Lab 5 — L'asynchrone

RabbitMQ : des événements qui voyagent, des services qui ne meurent plus ensemble

Dr. El Hadji Bassirou TOURE
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop

Année universitaire 2026–2027

Abstract

Dernier lab du fil rouge. Les événements du Lab 3 quittent la JVM : un **pont AMQP** les publie en JSON sur un exchange **RabbitMQ** ; `notifications` consomme depuis une **queue durable** ; et le module `paiement` déménage en **Python** (pika) — deuxième extraction du cours, cette fois *asynchrone* : ni façade, ni timeout, ni 503. La séance culmine avec la **démonstration miroir** de la panne du Lab 4 : `paiement-service` éteint, les confirmations *réussissent*, les messages *attendent*, et au redémarrage le service *rattrape tout* — zéro message perdu. En bonus : un webhook mobile money simulé, et la règle d'or qui va avec, l'**idempotence**.

Benefice

À l'issue de ce lab, vous saurez :

- déclarer exchange, queues durables et bindings, et lire l'UI de gestion de RabbitMQ ;
- publier depuis Spring (**Spring AMQP**) et consommer depuis Python (**pika**), avec l'ack au bon moment ;
- démontrer la garantie *at-least-once* — et payer son prix : l'**idempotence** ;
- trancher, intégration par intégration : synchrone ou asynchrone.

Point d'attention

Prérequis : dépôt au tag `lab5-depart` (= `lab4-final`) ; `./mvnw clean test` vert (10 juges) ; Docker fonctionnel ; internet (Maven : spring-boot-starter-amqp ; pip : pika).

Contents

1 Le constat : trois blessures, une racine **3**

2 Etape 1 — Le broker et le pont **3**
2.1 RabbitMQ dans l'orchestre . . . 3
2.2 Le pont : les faits partent en JSON . . . 4

3 Etape 2 — Les consommateurs **4**
3.1 Notifications : la queue durable . . . 5
3.2 Paiement demenage — sans facade . . . 5

4 Etape finale — La demonstration miroir **5**

5 ADR, livrable et cloture **6**

1 Le constat : trois blessures, une racine

Intuition

Confier un message à quelqu'un exige qu'il soit là. Le déposer dans une boîte aux lettres, non : la boîte garde, le destinataire relève quand il peut. Toute cette séance tient dans cette différence.

Relisez vos notes des labs 4 : la panne synchrone (503), le statut manuel, l'état évaporé. Discutez 10 minutes : pour chacune, la racine est-elle la même ? Conclusion attendue : nos services ne savent parler qu'**en direct et à l'instant**. Le remède s'appelle un **broker**.

Concept fondamental

RabbitMQ en quatre mots. Un *exchange* (le guichet de tri) reçoit chaque message avec sa *clé de routage* (`commande.confirmee`) et le copie dans les *queues durables* dont le *binding* correspond (`commande.*`). Le consommateur relève sa queue et *acquitte* après traitement — sans ack, le message reste. Queue durable + message persistant + ack = **at-least-once** : livré au moins une fois, à travers pannes et redémarrages.

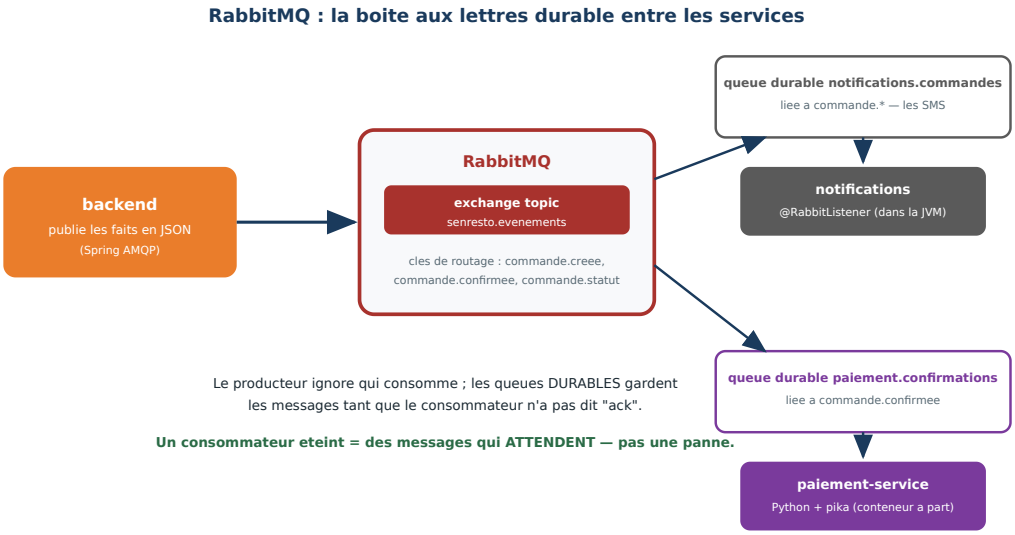


Figure 1: La cible : le broker au centre, deux queues durables, deux langages.

2 Etape 1 — Le broker et le pont

Qui pilote cette section ?

L'Huissier pilote compose et RabbitMQ. **Le Doyen** pilote le pont AMQP. **Le Rédacteur** prend possession de l'UI de gestion.

2.1 RabbitMQ dans l'orchestre

Dans `docker-compose.yml` :

```
rabbitmq:
  image: rabbitmq:3-management
  ports: ["5672:5672", "15672:15672"]   # AMQP + UI de gestion
```

Le backend en dépend (`depends_on`) et reçoit la variable `SPRING_RABBITMQ_HOST` valant `rabbitmq`. L'interface de gestion est sur le port `15672` (guest / guest) : votre poste d'observation pour toute la séance.

2.2 Le pont : les faits partent en JSON

Dépendance `spring-boot-starter-amqp`, puis deux briques dans un nouveau module `evenements/` (ou dans `commandes/interne` : discutez, c'est une ligne d'ADR) :

```
@Configuration
class ConfigurationAmqp {
    @Bean TopicExchange evenements() {
        return new TopicExchange("senresto.evenements", true, false);
    }
    @Bean Jackson2JsonMessageConverter json() {
        return new Jackson2JsonMessageConverter();   // records -> JSON
    }
}
```

```
@Component
class PontAmqp {
    private final RabbitTemplate rabbit;
    PontAmqp(RabbitTemplate rabbit) { this.rabbit = rabbit; }

    @EventListener
    void quandCreee(CommandeCreee evt)         { publier("commande.creee", evt); }
    @EventListener
    void quandConfirmee(CommandeConfirmee evt) { publier("commande.confirmee", evt); }
    @EventListener
    void quandStatut(CommandeStatutChangee evt){ publier("commande.statut", evt); }

    private void publier(String cle, Object evt) {
        rabbit.convertAndSend("senresto.evenements", cle, evt);
    }
}
```

Point cle a retenir

`CommandeMetier` n'a pas bougé d'une ligne : il publie ses faits comme au Lab 3. Le pont *écoute et réexpédie* — les frontières du modulith paient une dernière fois.

Point d'attention

Le réflexe du Lab 4, réappliqué : annotez `ConfigurationAmqp` et `PontAmqp` de `@Profile("amqp")` — sans broker (CI, `mvnw test`), le pont n'existe pas et les 10 juges restent verts ; compose active `amqp`. Même geste pour notifications à l'étape 2 : l'ancien `@EventListener` reste sous `@Profile("!amqp")` (le chemin historique du Strangler), le nouveau `@RabbitListener` vit sous `@Profile("amqp")`.

Tache a realiser

Compose à 5 conteneurs ; dans l'UI, créez une queue temporaire liée à `commande.#`, passez une commande : **regardez le JSON arriver**. 10 juges toujours verts (le pont ne casse rien).

Jalon Git

```
git tag lab5-etape1 && git push origin lab5-etape1
```

3 Etape 2 — Les consommateurs

Qui pilote cette section ?

Le Notaire migre notifications. **L'Appariteur** pilote la deuxième extraction : `paiement-service` en Python.

3.1 Notifications : la queue durable

Déclarez la queue et son binding (beans `Queue` et `Binding`, motif `commande.*`), puis remplacez les écouteurs internes d'`EcouteurNotifications` par :

```
@RabbitListener(queues = "notifications.commandes")
void surEvenement(Map<String, Object> evt,
        @Header("amqp_receivedRoutingKey") String cle) {
    switch (cle) {
        case "commande.creee" -> sms.envoyerSms(/* MEMES textes qu'avant */);
        // confirmee, statut : meme principe
    }
}
```

3.2 Paiement demenage — sans facade

Supprimez le module `paiement` du monolithe (il déménage — c'est la mort par famine du figuier : plus rien ne l'appelle), et créez `paiement-service/` (conteneur Python) :

```
# paiement-service/main.py -- pika, ~40 lignes
import json, os, pika

params = pika.ConnectionParameters(host=os.environ.get("RABBITMQ_HOST", "localhost"))
connexion = pika.BlockingConnection(params)
canal = connexion.channel()
canal.exchange_declare("senresto.evenements", "topic", durable=True)
canal.queue_declare("paiement.confirmations", durable=True)
canal.queue_bind("paiement.confirmations", "senresto.evenements",
                 "commande.confirmee")

deja_traites = set()          # l'idempotence, version simple

def sur_message(canal, methode, proprietes, corps):
    evt = json.loads(corps)
    if evt["commandeId"] in deja_traites:
        print(f"[PAIEMENT] {evt['numero']} deja traite, ignore", flush=True)
    else:
        deja_traites.add(evt["commandeId"])
        print(f"[PAIEMENT] Encaissement mobile money simule : "
              f"{evt['montantTotal']} FCFA pour la commande {evt['numero']}",
              flush=True)
    canal.basic_ack(methode.delivery_tag)     # l'ack APRES le traitement

canal.basic_consume("paiement.confirmations", sur_message)
print("[PAIEMENT] paiement-service en ecoute...", flush=True)
canal.start_consuming()
```

Point cle a retenir

Comparez au Lab 4 : cette extraction n'a exigé **ni interface, ni profils, ni timeouts, ni 503**. Personne n'attend la réponse du paiement : l'extraction asynchrone est radicalement moins chère. C'est LA leçon du cours : *le coût d'une frontière dépend de la nature de l'échange qui la traverse.*

Tache a realiser

6 conteneurs ; parcours complet : les `[SMS]` sortent du backend (via la queue), le `[PAIEMENT]` sort de **son** conteneur (`docker compose logs -f paiement-service`). 10 juges verts.

Jalon Git

```
git tag lab5-etape2 && git push origin lab5-etape2
```

4 Etape finale — La demonstration miroir

Qui pilote cette section ?

Tous ensemble, au vidéoprojecteur — c'est la scène finale du semestre, elle se joue en cinq actes.

1. `docker compose stop paiement-service` — le paiement est mort.

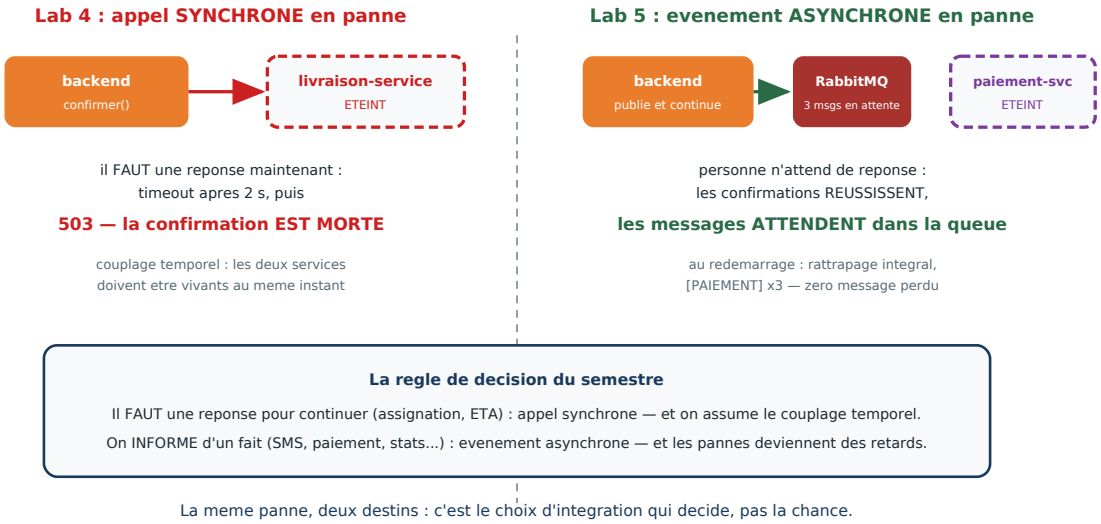


Figure 2: La même panne, deux destins.

2. Passez et confirmez **trois** commandes : tout réussit, **aucun 503** — comparez au Lab 4, même geste, autre destin.
3. L'UI de gestion : la queue `paiement.confirmations` affiche **3 messages prêts**, qui attendent.
4. `docker compose start paiement-service` : dans ses logs, le **rattrapage** — `[PAIEMENT]` ×3, dans l'ordre.
5. Bonus idempotence : dans l'UI, re-publiez à la main un des messages (onglet *Publish message*) : `deja traite, ignore`. Un webhook Wave rejoué ne doit *jamais* encaisser deux fois.

Point d'attention

Constatez aussi ce que l'asynchrone **ne répare pas** : l'assignation du livreur reste synchrone (il *faut* l'ETA dans la réponse), et les blessures « statut manuel » et « état évaporé » restent ouvertes — mais vous savez désormais *exactement* comment un événement `course.terminee` publié par le service Python les refermerait. C'est votre premier chantier d'après-cours.

Jalon Git

Démo des 5 actes réalisée et notée :

```
git tag lab5-final && git push origin lab5-final
```

5 ADR, livrable et cloture

L'**Architecte du jour** signe l'**ADR n°5 : « Synchrone ou asynchrone ? »**, dans `docs/adr/` : le tableau de *toutes* les intégrations de SénResto (assignation, SMS, paiement, positions, statuts) avec, pour chacune, le mode choisi et la justification — il faut une réponse, ou l'on informe d'un fait ?

Critère	Vérification
4 tags ; 10 juges verts	`git tag`, `mvnw clean test`
Le JSON observé dans l'UI ; queues durables liées	UI 15672
`[PAIEMENT]` depuis son conteneur	`compose logs`
La démo miroir : 3 attendus, 3 rattrapés, 0 perdu	notes + capture
Idempotence prouvée (message rejoué ignoré)	logs
ADR n°5 : le tableau complet des intégrations	`docs/adr/`

Clôture. En cinq séances, la même application est passée du God controller au système distribué polyglotte et résilient — sous tests à chaque pas, chaque décision tracée. Gardez ce dépôt : c'est votre portfolio d'architecte.

Fin du Lab 5 — et du fil rouge SénResto — Dr. El Hadji Bassirou TOURE — DMI/FST/UCAD